Beginner: Number Theory

BUBT-BAPS National Programming Camp 2025

A Fun Problem

Problem Statement:

Today is Friday. What day will it be after 100 days?

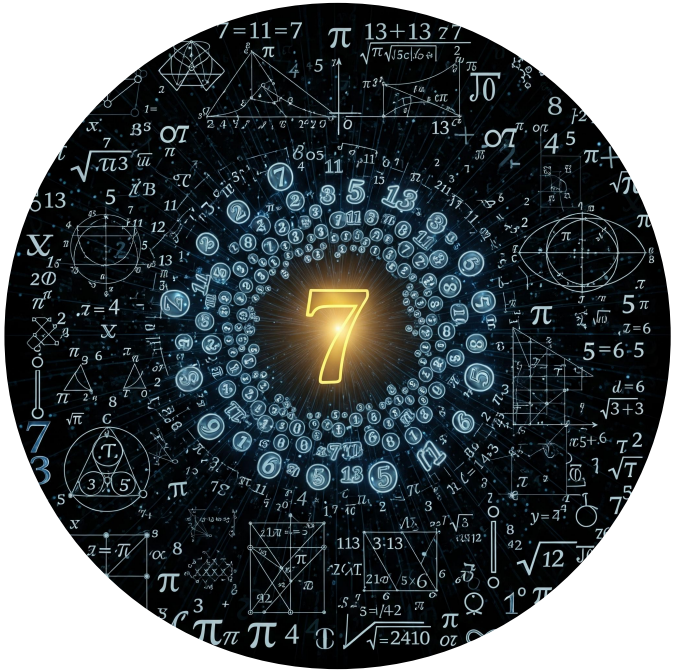
Solution:

- 100 % 7 = 2
- Friday + 2 $\Longrightarrow$ Sunday

What is Number Theory?

01 —— Number theory studies integers (whole numbers) and their properties

02 —— Crucial for modular arithmetic and other concepts



03 —— Essential for competitive programming algorithms

04 —— Helps solve problems involving prime numbers

Greatest Common Divisor (GCD)

The GCD of two integers is the largest positive integer that divides both numbers without a remainder.

Example: 6 is the GCD of 12 and 18 because there isn’t any number greater than 6 that divides both 12 and 18.

Question: How to calculate the GCD of any two numbers.

Do you remember the method to find the GCD?

18	30	1
	18	
12	18	1
	12	
6	12	2
	12	
	0	

Euclidean Algorithm for Computing GCD

The original Euclidean algorithm: subtract the smaller number from the larger one until one of the numbers is zero.

$$\gcd(a,b)=\begin{cases}\gcd(|a-b|,\min(a,b)), & a\neq b,\\ a, & a=b.\end{cases}$$

Then the algorithm is formulated in an extremely simple way:

$$\gcd(a,b)=\begin{cases}a, & \text{if } b=0,\\ \gcd(b,a \bmod b), & \text{otherwise.}\end{cases}$$

The simplest C++ implementation would be:

```
long long gcd(long long a, long long b) {
    return b == 0? a : gcd(b, a % b);
}
```

Some properties of GCD

- GCD is associative. So, gcd(a, b, c) = gcd(a, gcd(b, c)).
- When one of the numbers is zero, while the other is non-zero, their greatest common divisor, by definition, is the second number.
- When both numbers are zero, their greatest common divisor is undefined (it can be any arbitrarily large number), but it is convenient to define it as zero as well to preserve the associativity of gcd.
- Now we have a simple rule: if one of the numbers is zero, the greatest common divisor is the other number.

Least Common Multiple (LCM)

The Least Common Multiple (LCM) is the smallest positive integer that is a multiple of two or more integers.

Calculating the LCM can be reduced to calculating the GCD with the following simple formula:

$$lcm(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

A possible implementation that cleverly avoids integer overflows:

```
int lcm (int a, int b) {
    return a / gcd(a, b) * b;
}
```

Fadi and LCM

Problem Statement: Given a positive integer X (up to 10^{12}), find two positive integers a and b such that lcm(a, b) = X and max(a, b) is minimized. If multiple pairs satisfy the condition, any one is acceptable.

Analysis & Solution Path

- **Apply the Core Identity:** Start with $\text{lcm}(a, b) = (a \times b) / \gcd(a, b) = X$.
- **Constrain the Search:** For $\text{lcm}(a, b)$ to be X, both a and b must be divisors of X.
- **Connect Product and LCM:** The product $a \times b = \gcd(a, b) \times X$.
- **Minimize the Maximum:** To make a and b as close as possible, their product $a \times b$ should be minimized. This occurs when $\gcd(a, b) = 1$, making $a \times b = X$.
- **Transform the Problem:** Find two factors, a and b, of X whose product is X, $\gcd(a, b) = 1$ and that are as close as possible to each other.
- **Algorithmic Solution:** Iterate from $i = 1$ up to $\sqrt{X}$. For each divisor i, its corresponding factor is $j = X / i$. Check if $\gcd(i, j) == 1$. Keep track of the pair that minimizes $\max(i, j)$.

Prime Factorization

Prime factorization is the process of decomposing a composite number into a product of its prime factors.

This decomposition is unique for every integer greater than 1.

This process is a building block for many other number theory algorithms.

Question: How to perform prime factorization efficiently?

Sieve of Eratosthenes

~~1~~	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Sieve of Eratosthenes (A C++ Implementation)

```
bool prime[n + 1];
memset(prime, true, sizeof(prime)); // true if prime

for(int i = 2; i <= n; i++) {
    if(prime[i] == true && (long long) i * i <= n) {
        for(int j = 2 * i; j <= n; j += i) {
            prime[j] = false;
        }
    }
}
```

Time Complexity: O(n log n)

Sieve of Eratosthenes (More optimized)

```
bool prime[n + 1];
memset(prime, true, sizeof(prime)); // true if prime

for(int i = 4; i <= n; i += 2) prime[i] = false;

for(int i = 3; i * i <= n; i += 2) {
    if(prime[i] == true) {
        for(int j = i * i; j <= n; j += 2 * i) {
            prime[j] = false;
        }
    }
}
```

Time Complexity: O(n log log n)

More to learn: Segmented Sieve, Memory Optimized Sieve

How to perform prime factorization efficiently?

```
vector<long long> primes; //this holds all the primes upto n
vector<long long> getFactorization(long long n) {
    vector<long long> factorization;
    for(long long d : primes) {
        if(d * d > n) break;
        while(n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if(n > 1) factorization.push_back(n);
    return factorization;
}
```

Time Complexity per query: O(sqrt(n)). **Can you do it more efficiently? Hint: SPF**

Divisors

Problem Statement: Find all integers N in the range $[1,10^6]$ for which d(N), the number of positive divisors of N, is equal to a product of two distinct primes, $p \times q$.

Analysis & Solution Path

Analyze Constraints: The range is up to 10^6. An O(N) approach per number is too slow, confirming the need for precomputation.
Identify Core Task: We need the number of divisors, d(N), for every N from 1 to 10^6.
Sieve for NOD: We can compute d(N) for all N in the range using an O(NlogN) sieve:

- Initialize an array `nod[1...10^6]` with zeros.
- For `i` from 1 to 10^6, iterate through its multiples `j` and increment `nod[j]`.

Check the Property: Once the `nod` array is populated, we need an efficient way to check if a number `k = nod[N]` is a product of two distinct primes. This can be done with a precomputed primality test (another sieve).
Final Algorithm:

- Sieve for primes up to 10^6.
- Sieve for the number of divisors (`nod` array).
- Iterate N from 1 to 10^6, check `nod[N]`, and store valid N.

Counting and Summing Divisors (NOD & SOD)

Let the prime factorization of n be: $n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$

Number of Divisors (NOD):
The total number of distinct divisors is the product of (exponent + 1) for each prime factor.

$$d(n) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

Sum of Divisors (SOD):
The sum of all positive divisors is the product of the sum of powers for each prime factor. Each sum is a geometric series.

$$\sigma(n) = \prod_{i=1}^{k} (p_i^0 + p_i^1 + \cdots + p_i^{a_i}) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

A Way of Thinking About NOD

- If there is only one distinct prime divisor $n = p_1^{e1}$, then there are obviously $e_1 + 1$ divisors $(1, p_1, p_1^2, \ldots, p_1^{e1})$.
- If there are two distinct prime divisors $n = p_1^{e1} \times p_2^{e2}$, then you can arrange all divisors in the form of a table.
- So the number of divisors is trivially $(e_1 + 1) \times (e_2 + 1)$.

A similar argument can be made if there are more than two distinct prime factors.

	1	p_2	p_2^2	$\cdots$	$p_2^{e_2}$
1	1	p_2	p_2^2	$\cdots$	$p_2^{e_2}$
p_1	p_1	$p_1 \cdot p_2$	$p_1 \cdot p_2^2$	$\cdots$	$p_1 \cdot p_2^{e_2}$
p_1^2	p_1^2	$p_1^2 \cdot p_2$	$p_1^2 \cdot p_2^2$	$\cdots$	$p_1^2 \cdot p_2^{e_2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$p_1^{e_1}$	$p_1^{e_1}$	$p_1^{e_1} \cdot p_2$	$p_1^{e_1} \cdot p_2^2$	$\cdots$	$p_1^{e_1} \cdot p_2^{e_2}$

Sum of Divisors (SOD)

We can use the same argument as the previous section.
If there is only one distinct prime divisor $n = p_1^{e1}$, then the sum is:

$$1 + p_1 + p_1^2 + \cdots + p_1^{e_1} = \frac{p_1^{e_1+1} - 1}{p_1 - 1}$$

If there are two distinct prime divisors $n = p_1^{e1} \times p_2^{e2}$, then we can make the same table as before. The only difference is that now we want to compute the sum instead of counting the elements. It is easy to see, that the sum of each combination can be expressed as:

$$\left(1 + p_1 + p_1^2 + \cdots + p_1^{e_1}\right) \cdot \left(1 + p_2 + p_2^2 + \cdots + p_2^{e_2}\right) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1}$$

In general, for $n = p_1^{e1} \times p_2^{e2} \ldots p_k^{ek}$ we receive the formula:

$$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \cdots \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

DIVSUM2

Problem Statement: Given a natural number n ($1 \le n \le 10^{16}$), please output the summation of all its proper divisors.

Definition: A proper divisor of a natural number is the divisor that is strictly less than the number.

e.g. number 20 has 5 proper divisors: 1, 2, 4, 5, 10, and the divisor summation is: 1 + 2 + 4 + 5 + 10 = 22.

Hint: What if you calculate SOD at first?

The World of Modulo: Euler's Totient Function (φ)

Definition:

$\phi(n)$ counts the number of positive integers k in the range $1 \le k \le n$ that are relatively prime (coprime) to n, i.e., $\gcd(k, n) = 1$.

Properties & Formula:

- If p is prime, $\phi(p) = p - 1$.
- If p is prime, $\phi(p^k) = p^k - p^{k-1}$.
- If $\gcd(a, b) = 1$, then $\phi(a \cdot b) = \phi(a) \cdot \phi(b)$.

For $n = p_1^{a1} \cdots p_k^{ak}$, the formula is: $\phi(n) = n \cdot \prod_{i=1}^{k}(1 - \frac{1}{p_i})$

Calculation Methods: Can you use the **sieve** method to compute $\phi(i)$ for all i from 1 to n? How?

Same GCDs

Problem Statement: Given two integers a and m (with $1 \le a < m \le 10^{10}$), count the number of integers x such that $0 \le x < m$ and $\gcd(a, m) = \gcd(a + x, m)$.

Analysis & Solution Path (1)

Simplify: Let g = gcd(a, m). The condition is gcd(a + x, m) = g.

Deduce Property of x: For g to divide a + x (since it already divides a and m), g must also divide x. So, x must be a multiple of g. Let x = k · g.

Transform the Equation:

$$gcd(a + k \cdot g, m) = g$$

Factor out g:

$$g \cdot gcd(\frac{a}{g} + k, \frac{m}{g}) = g$$

This simplifies to:

$$gcd(\frac{a}{g} + k, \frac{m}{g}) = 1$$

Analysis & Solution Path (2)

Reframe the Problem: Let $a' = a / g$ and $m' = m / g$. We need to count integers k such that $\gcd(a' + k, m') = 1$.

Determine Range of k: $0 \le x < m \Rightarrow 0 \le k \cdot g < m \Rightarrow 0 \le k < m / g = m'$.

Connect to Euler's Totient: As k ranges from 0 to $m' - 1$, the term $a' + k$ cycles through a complete set of residues modulo m'. The number of integers in this set coprime to m' is, by definition, $\phi(m')$.

Final Answer: The count is $\phi(m / g) = \phi(m / \gcd(a, m))$.

The Art of Modular Division

In modular arithmetic, addition, subtraction, and multiplication are straightforward.

- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$
- $(a * b) \bmod m = ((a \bmod m) * (b \bmod m)) \bmod m$

Division, however, is not directly defined. The operation $(a / b) \pmod m$ is handled by finding a modular multiplicative inverse for b and performing multiplication instead.

Modular Multiplicative Inverse

Definition: The modular multiplicative inverse of an integer a modulo m is an integer x such that their product is congruent to 1.

$a \cdot x \equiv 1 \pmod{m}$

This inverse is often denoted as a^{-1}.

Existence

A modular inverse for a modulo m exists if and only if a and m are coprime, i.e., $\gcd(a, m) = 1$.

Modular Inverse - The Three Methods

Extended Euclidean Algorithm (O(log m)):

- Finds x and y such that $ax + my = \gcd(a, m)$.
- If $\gcd(a, m) = 1$, then $ax + my = 1$. Modulo m, this becomes $ax \equiv 1 \pmod{m}$.
- This is the most general method, working for any modulus m (prime or composite).

Fermat's Little Theorem (O(log m)):

- Applicable only when the modulus m is a prime number, say p.
- If p does not divide a, then $a^{p-1} \equiv 1 \pmod{p}$.
- This means the inverse is $\mathbf{a^{p-2} \pmod{p}}$, calculated with **modular exponentiation**.

Euler's Totient Theorem (O(√m + log m)):

- Generalizes Fermat's Little Theorem to any modulus m.
- If $\gcd(a, m) = 1$, then $a^{\phi(m)} \equiv 1 \pmod{m}$.
- The inverse is $a^{\phi(m)-1} \pmod{m}$.

Binomial Coefficients

Problem Statement:

Calculate $nCr = \frac{n!}{r!(n-r)!}$ modulo a large prime p. Assume n can be large (e.g., up to 10^6), so precomputation is feasible.

Analysis and Solution Path

Challenge: Direct calculation of factorials will cause overflow. All calculations must be modulo p.

Handle Division: Division modulo p requires multiplication by the modular inverse. The formula becomes: $nCr \equiv n! \times (r!)^{-1} \times ((n-r)!)^{-1} \pmod{p}$

Find Inverses: Since p is prime, use Fermat's Little Theorem. The inverse of k! is $(k!)^{p-2} \pmod{p}$.

Implementation Strategy:

- **Precomputation:** Precompute factorials modulo p in an array in O(n) time.
- **Modular Exponentiation:** Implement a function power(base, exp, mod) to calculate modular powers in O(log exp) time.
- **Combine:** Use the precomputed factorials and the power function to calculate the final result.

Modular Exponentiation (Binary Exponentiation)

The Problem:

Calculating Large Powers Calculating expressions like a^b (mod m) is a common requirement, especially when finding modular inverses. A naive approach of multiplying a by itself b times is too slow, with a complexity of O(b). When b is large (e.g., up to 10^{18}), this is infeasible.

The Solution: Binary Exponentiation

Binary Exponentiation (also known as exponentiation by squaring) is an efficient algorithm that computes a^b (mod m) in O(log b) time. It works by repeatedly squaring the base and using the binary representation of the exponent b to decide when to multiply the result.

- Core Idea: If the exponent b is even, $a^b = (a^{b/2})^2$. If b is odd, $a^b = a \cdot a^{b-1}$.

Binary Exponentiation (A C++ Implementation)

```
// Calculates (base^exp) % mod in O(log exp)
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        // If exp is odd, multiply base with result
        if (exp % 2 == 1) {
            res = (res * base) % mod;
        }
        // exp must be even now, square the base
        base = (base * base) % mod;
        exp /= 2; // Halve the exponent
    }
    return res;
}
```

More Practice Problems

GCD/LCM:

- Orac and LCM (Codeforces 1349A)
- GCD vs LCM (Codeforces 1665A)

Factorization/Divisors:

- Integer Factorization (SPOJ FACT0, FACT1, FACT2)
- Sum of Divisors (CSES)

Euler's Totient/Advanced:

- Coprime Integers (Kattis)
- Product 1 Modulo N (Codeforces 1514C)

Modular Inverse/Combinatorics:

- Creating Strings II (CSES)
- Exponentiation II (CSES)

Reference

- CP Algorithm
- Wikipedia
- প্রোগ্রামিং কনটেস্ট - ডেটা স্ট্রাকচার ও এলগোরিদম (মো: মাহবুবুল হাসান)
- Codeforces, SPOJ, Kattis, CSES

Thank You